

Android 第 3 章 —— 3 Button、ImageView 及计算器案例教学讲义

各位同学，今天咱们聚焦三个核心内容：Button（按钮）、ImageView（图像显示）、ImageButton（图像按钮），还会教大家怎么同时展示文本和图像，最后用一个计算器案例把这些知识串起来，全程大白话，保证大家听得懂、用得上。

一、Button：最常用的“互动开关”

Button（按钮）是用户和 APP 互动的核心控件，比如“登录”“提交”“返回”按钮都靠它。大家可能不知道，Button 其实是从 TextView 派生出来的，也就是说它天生就有 TextView 的很多属性（比如设置文本、颜色），还多了“点击互动”的能力。

1. Button 的两个基础属性：textAllCaps 和 onClick

- **textAllCaps**：控制按钮上的文字是否自动变成大写。默认是 `true`（自动大写），比如你写“login”，按钮上会显示“LOGIN”；如果想让文字保持原样，就设置 `android:textAllCaps="false"`。
- **onClick**：直接在布局文件里绑定点击事件（简单场景用）。比如想让按钮点击后执行 `clickBtn` 方法，就在布局里写 `android:onClick="clickBtn"`，然后在 Java 代码里定义这个方法：

```
// 方法名要和布局里的onClick值一致
public void clickBtn(View view) {
    // 按钮被点击后要做的事，比如弹提示
    Toast.makeText(this, "按钮被点击啦", Toast.LENGTH_SHORT).show();
}
```

2. 按钮的两种点击事件：普通点击和长按点击

按钮有两种常用互动方式：点一下（普通点击）、按住不放（长按点击），咱们分别说怎么实现。

（1）普通点击事件（常用）

除了上面用 `onClick` 的简单方式，还有一种更灵活的“代码绑定”方式，适合复杂逻辑，步骤如下：

1. 在布局文件里给按钮设个 ID: `android:id="@+id/myBtn"`。
2. 在 Java 代码里 “找到” 这个按钮，然后绑定点击事件：

```
// 1. 找到按钮（onCreate方法里写）
Button myBtn = findViewById(R.id.myBtn);
// 2. 绑定点击事件
myBtn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        // 点击后执行的逻辑，比如跳页面、改文字
        myBtn.setText("我被点过了");
    }
});
```

(2) 长按点击事件

长按点击就是用户按住按钮 1 秒左右触发的事件，实现方式和普通点击类似，只是换了个方法：

```
myBtn.setOnLongClickListener(new View.OnLongClickListener() {
    @Override
    public boolean onLongClick(View v) {
        // 长按后执行的逻辑，比如弹确认框
        Toast.makeText(MainActivity.this, "你长按我啦", Toast.LENGTH_SHORT).show();
        // 这里返回true表示“消费”了长按事件，不会再触发普通点击；返回false会先触发长按，再
        触发普通点击
        return true;
    }
});
```

3. 禁用和恢复按钮：enabled 属性

有时候咱们需要让按钮 “失效”（比如用户没填完表单，登录按钮不能点），这时候就用 `enabled` 属性：

- **禁用按钮**：`myBtn.setEnabled(false);`，按钮会变成灰色，不能点击。
- **恢复按钮**：`myBtn.setEnabled(true);`，按钮恢复正常颜色，能点击。

也可以在布局文件里直接设置初始状态：`android:enabled="false"`（默认是 `true`）。

二、ImageView：专门“展示图片”的控件

ImageView 就是用来显示图片的，比如 APP 的图标、轮播图、商品图片都靠它。咱们重点学怎么给它设图片、怎么控制图片缩放。

1. 两种设置图片的方式：XML 和 Java

给 ImageView 放图片，有两种常用方法，根据场景选：

(1) XML 里设置：android:src

最简单的方式，直接在布局文件里写 `android:src="@drawable/图片名"`，前提是图片已经放在 `res/drawable` 文件夹里（比如图片叫 `cat.png`，就写 `@drawable/cat`）。

示例代码：

```
<ImageView
    android:id="@+id/myImg"
    android:layout_width="200dp"
    android:layout_height="200dp"
    android:src="@drawable/cat"/> <!-- 直接引用drawable里的图片 -->
```

(2) Java 代码里设置：setImageResource

有时候需要动态换图（比如点击按钮换图片），就用 Java 代码：

```
// 1. 找到ImageView
ImageView myImg = findViewById(R.id.myImg);
// 2. 动态设置图片（比如点击按钮后换图）
myBtn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        myImg.setImageResource(R.drawable.dog); // 换成dog.png
    }
});
```

2. 控制图片缩放：scaleType 属性

有时候图片尺寸和 ImageView 的尺寸不匹配（比如图片太大，ImageView 装不下），这时候就用 `scaleType` 控制图片怎么缩放，常用的值有这些，咱们用大白话解释：

- **fitCenter**：默认值，图片按比例缩放，完整显示在 ImageView 里，居中对齐（不会变形，可能有空白）。
- **fitXY**：图片拉伸填满整个 ImageView，不管比例（可能变形，比如正方形图片拉成长方形）。
- **centerCrop**：图片按比例缩放，填满 ImageView，超出部分切掉（不会变形，能填满，但可能看不到图片边缘）。
- **centerInside**：图片按比例缩放，完整显示在 ImageView 里，比 fitCenter 更灵活（如果图片比 ImageView 小，就居中显示不缩放）。

设置方式很简单，在布局里加 `android:scaleType="fitCenter"`（换成你需要的类型）就行。

三、ImageButton：能“点击的图片”

ImageButton 其实是 Button 的子类，它和 ImageView 的区别是：ImageButton 可以点击（有按钮的互动效果，比如点击时变灰），而 ImageView 默认不能点击；另外 ImageButton 可以分别设置“前景图”和“背景图”。

1. ImageButton 的核心特点：前景 + 背景，可缩放

- **前景图**：就是按钮上的主要图片，用 `android:src` 设置（和 ImageView 一样），支持缩放（靠 `scaleType`）。
- **背景图**：就是按钮的背景（比如按钮的形状、颜色），用 `android:background` 设置，可以是颜色或图片。

示例：做一个带背景的图像按钮，点击有效果：

```
<ImageButton
    android:layout_width="100dp"
    android:layout_height="100dp"
    android:src="@drawable/ic_login" <!-- 前景图：登录图标 -->
    android:background="@drawable/btn_bg" <!-- 背景图：按钮的圆角背景 -->
    android:scaleType="fitCenter"/> <!-- 前景图按比例缩放 -->
```

这里的btn_bg可以是一个 XML 文件（定义圆角、颜色），比如在 drawable 里建btn_bg.xml：

```
<shape xmlns:android="http://schemas.android.com/apk/res/android"
    android:shape="rectangle">
    <solid android:color="#FF0000"/> <!-- 背景色红色 -->
    <corners android:radius="10dp"/> <!-- 圆角10dp -->
</shape>
```

这样 ImageButton 就有红色圆角背景，上面放登录图标，点击时背景还会变灰。

四、同时展示文本和图像：两种常用方案

有时候咱们需要“文字+图片”的组合（比如“返回首页”按钮，左边图片右边文字），有两种简单方案，按需选择。

方案 1：用 LinearLayout 组合 ImageView 和 TextView

这是最灵活的方式，把图片和文字分别放在 ImageView 和 TextView 里，再用 LinearLayout 把它们排成一行或一列。

示例：左边图片、右边文字的“返回”按钮：

```
<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal" <!-- 水平排列：图片在左，文字在右 -->
    android:padding="10dp"> <!-- 加内边距，避免太挤 -->
    <!-- 图片 -->
    <ImageView
        android:layout_width="24dp"
        android:layout_height="24dp"
        android:src="@drawable/ic_back"/>
    <!-- 文字 -->
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="返回首页"
        android:textSize="16sp"
```

```
android:layout_marginLeft="5dp"/> <!-- 图片和文字之间留5dp距离 -->
</LinearLayout>
```

如果想让这个组合能点击，给 LinearLayout 加个点击事件就行（和 Button 点击事件一样）。

方案 2：用 Button 的 drawableXXX 属性（更简单）

Button 有四个特殊属性，可以直接在文字的上下左右放图片，不用组合控件，更简单：

- `android:drawableLeft`：图片在文字左边。
- `android:drawableRight`：图片在文字右边。
- `android:drawableTop`：图片在文字上边。
- `android:drawableBottom`：图片在文字下边。

还可以用 `android:drawablePadding` 设置图片和文字的距离。

示例：文字左边放图片的“提交”按钮：

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="提交"
    android:textSize="16sp"
    android:drawableLeft="@drawable/ic_submit" <!-- 文字左边放图片 -->
    android:drawablePadding="5dp"/> <!-- 图片和文字间距5dp -->
```

这种方式代码更简洁，适合简单的“文字 + 图片”按钮。

五、实战案例：做一个简单的计算器

学完上面的控件，咱们用一个计算器案例把知识串起来，计算器需要两部分：界面布局（用之前学的布局和控件）、逻辑处理（点击按钮计算）。

1. 计算器界面布局：用 GridLayout

计算器的按键是整齐的网格（比如 4 列，多行），所以用 GridLayout 最合适，界面包含两部分：

- 顶部：TextView（显示输入的数字和结果）。

- 底部：GridLayout（放数字键、运算符键）。

布局代码示例：

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <!-- 顶部显示区域：TextView，右对齐，字体大一点 -->
    <TextView
        android:id="@+id/resultTv"
        android:layout_width="match_parent"
        android:layout_height="100dp"
        android:background="#F5F5F5"
        android:gravity="right|center_vertical" <!-- 文字右对齐，垂直居中 -->
        android:paddingRight="20dp"
        android:textSize="28sp"
        android:text="0"/> <!-- 默认显示0 -->
    <!-- 底部按键区域：GridLayout，4列 -->
    <GridLayout
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" <!-- 占满剩余高度 -->
        android:columnCount="4" <!-- 4列 -->
        android:rowCount="5"> <!-- 5行 -->
        <!-- 第一行：C（清空）、±（正负）、%（百分比）、÷（除） -->
        <Button android:text="C" android:id="@+id/btnC"/>
        <Button android:text="±" android:id="@+id/btnSign"/>
        <Button android:text="%" android:id="@+id/btnPercent"/>
        <Button android:text="÷" android:id="@+id/btnDivide" android:background="#FF9800"/>
    >
    <!-- 第二行：7、8、9、×（乘） -->
    <Button android:text="7" android:id="@+id/btn7"/>
    <Button android:text="8" android:id="@+id/btn8"/>
    <Button android:text="9" android:id="@+id/btn9"/>
    <Button android:text="×" android:id="@+id/btnMultiply" android:background="#FF9800"/>
    <!-- 第三行：4、5、6、-（减） -->
    <Button android:text="4" android:id="@+id/btn4"/>
    <Button android:text="5" android:id="@+id/btn5"/>
    <Button android:text="6" android:id="@+id/btn6"/>
```

```
<Button android:text="-" android:id="@+id/btnMinus" android:background="#FF9800"/>
<!-- 第四行：1、2、3、+（加） -->
<Button android:text="1" android:id="@+id/btn1"/>
<Button android:text="2" android:id="@+id/btn2"/>
<Button android:text="3" android:id="@+id/btn3"/>
<Button android:text="+" android:id="@+id/btnPlus" android:background="#FF9800"/>
<!-- 第五行：0（占2列）、.（小数点）、=（等于） -->
<Button android:text="0" android:id="@+id/btn0" android:layout_columnSpan="2"/> <!-- 占2列 -->
<Button android:text="." android:id="@+id/btnDot"/>
<Button android:text="=" android:id="@+id/btnEqual" android:background="#FF9800"/>
</GridLayout>
</LinearLayout>
```

这里给运算符按钮设了橙色背景，和数字键区分开，0 键占 2 列，符合常见计算器的布局。

2. 计算器逻辑处理：点击按钮实现计算

逻辑处理的核心是：点击数字键显示数字，点击运算符记录运算，点击等号计算结果，点击 C 清空。步骤如下：

（1）定义变量存储数据

在 Activity 里定义几个变量，用来存输入的数字、运算符、计算结果：

```
private TextView resultTv;
private String firstNum = ""; // 第一个操作数
private String operator = ""; // 运算符（+、-、×、÷）
private String secondNum = ""; // 第二个操作数
private boolean isOperatorClicked = false; // 标记是否点击了运算符
```

（2）找到所有按钮，绑定点击事件

在 onCreate 方法里找到所有按钮，给数字键、运算符键、功能键分别写点击逻辑：

```
// 1. 找到显示区域
resultTv = findViewById(R.id.resultTv);
// 2. 数字键点击逻辑（0-9、.）
```



```
findViewById(R.id.btn0).setOnClickListener(v -> setNum("0"));
findViewById(R.id.btn1).setOnClickListener(v -> setNum("1"));
// （省略btn2-btn9、btnDot的点击绑定，都是调用setNum方法，传对应的字符）
findViewById(R.id.btnDot).setOnClickListener(v -> setNum("."));
// 3. 运算符点击逻辑（+、-、×、÷）
findViewById(R.id.btnPlus).setOnClickListener(v -> setOperator("+"));
findViewById(R.id.btnMinus).setOnClickListener(v -> setOperator("-"));
findViewById(R.id.btnMultiply).setOnClickListener(v -> setOperator("×"));
findViewById(R.id.btnDivide).setOnClickListener(v -> setOperator("÷"));
// 4. 功能键点击逻辑（C、±、%、=）
findViewById(R.id.btnC).setOnClickListener(v -> clear());
findViewById(R.id.btnSign).setOnClickListener(v -> changeSign());
findViewById(R.id.btnPercent).setOnClickListener(v -> setPercent());
findViewById(R.id.btnEqual).setOnClickListener(v -> calculate());
```

(3) 实现各个逻辑方法

接下来实现上面调用的`setNum`（数字输入）、`setOperator`（选运算符）、`calculate`（计算）等方法：

① `setNum`：处理数字和小数点输入

```
private void setNum(String num) {
    if (isOperatorClicked) {
        // 如果点击过运算符，输入的是第二个操作数
        secondNum += num;
        resultTv.setText(secondNum);
    } else {
        // 没点击运算符，输入的是第一个操作数（避免开头是0，比如输入0后再输0，还是显示0）
        if (firstNum.equals("0") && !num.equals(".")) {
            firstNum = num;
        } else {
            firstNum += num;
        }
        resultTv.setText(firstNum);
    }
}
```

②

| (注：文档部分内容可能由 AI 生成)